SQL Code

```sql
/* ============================================================
   GYM MEMBERSHIP AND MANAGEMENT SYSTEM
   Part B: Database Implementation

   Section 1: DROP EXISTING TABLES
   Removes the previous assignment tables before recreating them.
   ============================================================ */

DROP TABLE PAYMENT CASCADE CONSTRAINTS;
DROP TABLE CLASS_BOOKING CASCADE CONSTRAINTS;
DROP TABLE FITNESS_CLASS CASCADE CONSTRAINTS;
DROP TABLE MEMBERSHIP CASCADE CONSTRAINTS;
DROP TABLE TRAINER CASCADE CONSTRAINTS;
DROP TABLE MEMBERSHIP_PLAN CASCADE CONSTRAINTS;
DROP TABLE MEMBER CASCADE CONSTRAINTS;


/* ============================================================
   Section 2: CREATE TABLES
   Identity columns generate primary key values automatically.
   ============================================================ */


/* 1. MEMBER */

CREATE TABLE MEMBER (
    member_id NUMBER GENERATED ALWAYS AS IDENTITY,
    full_name VARCHAR2(100) NOT NULL,
    email VARCHAR2(100),
    phone VARCHAR2(20),
    join_date DATE DEFAULT SYSDATE NOT NULL,
    city VARCHAR2(50),

    CONSTRAINT pk_member PRIMARY KEY (member_id),
    CONSTRAINT uq_member_email UNIQUE (email)
);


/* 2. MEMBERSHIP_PLAN */

CREATE TABLE MEMBERSHIP_PLAN (
    plan_id NUMBER GENERATED ALWAYS AS IDENTITY,
    plan_name VARCHAR2(100) NOT NULL,
    duration_months NUMBER NOT NULL,
    price NUMBER(10,2) NOT NULL,

    CONSTRAINT pk_membership_plan PRIMARY KEY (plan_id),
    CONSTRAINT ck_plan_duration CHECK (duration_months > 0),
    CONSTRAINT ck_plan_price CHECK (price >= 0)
);
```

```sql


/* 3. MEMBERSHIP */

CREATE TABLE MEMBERSHIP (
    membership_id NUMBER GENERATED ALWAYS AS IDENTITY,
    member_id NUMBER NOT NULL,
    plan_id NUMBER NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    membership_status VARCHAR2(20) NOT NULL,

    CONSTRAINT pk_membership PRIMARY KEY (membership_id),
    CONSTRAINT fk_membership_member
        FOREIGN KEY (member_id) REFERENCES MEMBER(member_id),
    CONSTRAINT fk_membership_plan
        FOREIGN KEY (plan_id) REFERENCES MEMBERSHIP_PLAN(plan_id),
    CONSTRAINT ck_membership_status
        CHECK (membership_status IN ('Active', 'Expired', 'Pending')),
    CONSTRAINT ck_membership_dates
        CHECK (end_date IS NULL OR end_date >= start_date)
);


/* 4. TRAINER */

CREATE TABLE TRAINER (
    trainer_id NUMBER GENERATED ALWAYS AS IDENTITY,
    full_name VARCHAR2(100) NOT NULL,
    specialization VARCHAR2(100),
    hire_date DATE NOT NULL,

    CONSTRAINT pk_trainer PRIMARY KEY (trainer_id)
);


/* 5. FITNESS_CLASS */

CREATE TABLE FITNESS_CLASS (
    class_id NUMBER GENERATED ALWAYS AS IDENTITY,
    class_name VARCHAR2(100) NOT NULL,
    trainer_id NUMBER,
    capacity NUMBER NOT NULL,
    scheduled_at TIMESTAMP NOT NULL,

    CONSTRAINT pk_fitness_class PRIMARY KEY (class_id),
    CONSTRAINT fk_class_trainer
        FOREIGN KEY (trainer_id) REFERENCES TRAINER(trainer_id),
    CONSTRAINT ck_class_capacity CHECK (capacity > 0)
);


/* 6. CLASS_BOOKING */
```

```sql

CREATE TABLE CLASS_BOOKING (
    booking_id NUMBER GENERATED ALWAYS AS IDENTITY,
    member_id NUMBER NOT NULL,
    class_id NUMBER NOT NULL,
    booking_date DATE DEFAULT SYSDATE NOT NULL,
    attendance_status VARCHAR2(20) NOT NULL,

    CONSTRAINT pk_class_booking PRIMARY KEY (booking_id),
    CONSTRAINT fk_booking_member
        FOREIGN KEY (member_id) REFERENCES MEMBER(member_id),
    CONSTRAINT fk_booking_class
        FOREIGN KEY (class_id) REFERENCES FITNESS_CLASS(class_id),
    CONSTRAINT ck_attendance_status
        CHECK (attendance_status IN
            ('Booked', 'Attended', 'Absent', 'Cancelled'))
);


/* 7. PAYMENT */

CREATE TABLE PAYMENT (
    payment_id NUMBER GENERATED ALWAYS AS IDENTITY,
    membership_id NUMBER NOT NULL,
    payment_date DATE DEFAULT SYSDATE NOT NULL,
    amount NUMBER(10,2) NOT NULL,
    payment_method VARCHAR2(30) NOT NULL,
    payment_status VARCHAR2(20) NOT NULL,

    CONSTRAINT pk_payment PRIMARY KEY (payment_id),
    CONSTRAINT fk_payment_membership
        FOREIGN KEY (membership_id)
        REFERENCES MEMBERSHIP(membership_id),
    CONSTRAINT ck_payment_amount CHECK (amount >= 0),
    CONSTRAINT ck_payment_method
        CHECK (payment_method IN
            ('Cash', 'Card', 'Bank Transfer')),
    CONSTRAINT ck_payment_status
        CHECK (payment_status IN
            ('Successful', 'Pending', 'Failed'))
);


/* ============================================================
   End of Section 2: CREATE TABLES
   Next: INSERT SAMPLE DATA
   ============================================================ */

/* ============================================================
   GYM MEMBERSHIP AND MANAGEMENT SYSTEM
   PART B: DATABASE IMPLEMENTATION

   SECTION 3: INSERT SAMPLE DATA
```

```
   Four records are inserted into each table.
   Primary key IDs are generated automatically by Oracle.

   Insertion order:
   1. MEMBER
   2. MEMBERSHIP_PLAN
   3. TRAINER
   4. MEMBERSHIP
   5. FITNESS_CLASS
   6. CLASS_BOOKING
   7. PAYMENT
   ============================================================ */


/* ============================================================
   1. MEMBER
   Insert four gym members.
   Member 4 will have no memberships.
   ============================================================ */

INSERT INTO MEMBER (full_name, email, phone, join_date, city)
VALUES ('Ahmad Khaled', 'ahmad@example.com', '0791111111',
        DATE '2025-01-10', 'Amman');

INSERT INTO MEMBER (full_name, email, phone, join_date, city)
VALUES ('Sara Omar', 'sara@example.com', '0782222222',
        DATE '2025-02-15', 'Zarqa');

INSERT INTO MEMBER (full_name, email, phone, join_date, city)
VALUES ('Lina Samir', 'lina@example.com', NULL,
        DATE '2025-03-20', 'Amman');

INSERT INTO MEMBER (full_name, email, phone, join_date, city)
VALUES ('Omar Nasser', NULL, NULL,
        DATE '2025-04-05', 'Irbid');


/* ============================================================
   2. MEMBERSHIP_PLAN
   Insert four plans.
   Plan 4 will have no memberships.
   ============================================================ */

INSERT INTO MEMBERSHIP_PLAN
    (plan_name, duration_months, price)
VALUES ('Monthly', 1, 40);

INSERT INTO MEMBERSHIP_PLAN
    (plan_name, duration_months, price)
VALUES ('Quarterly', 3, 105);

INSERT INTO MEMBERSHIP_PLAN
```

```sql
    (plan_name, duration_months, price)
VALUES ('Annual', 12, 360);

INSERT INTO MEMBERSHIP_PLAN
    (plan_name, duration_months, price)
VALUES ('Student', 6, 150);


/* ============================================================
   3. TRAINER
   Insert four trainers.
   Trainer 4 will have no assigned classes.
   ============================================================ */

INSERT INTO TRAINER (full_name, specialization, hire_date)
VALUES ('Khaled Mahmoud', 'Strength Training',
        DATE '2022-03-01');

INSERT INTO TRAINER (full_name, specialization, hire_date)
VALUES ('Rana Fadi', 'Yoga',
        DATE '2023-06-15');

INSERT INTO TRAINER (full_name, specialization, hire_date)
VALUES ('Samer Ali', 'Cardio',
        DATE '2024-01-10');

INSERT INTO TRAINER (full_name, specialization, hire_date)
VALUES ('Huda Basel', 'Pilates',
        DATE '2024-08-20');


/* ============================================================
   4. MEMBERSHIP
   Insert four memberships.
   Member 1 has two memberships.
   Member 4 has no membership.
   Membership 4 will have no payments.
   ============================================================ */

INSERT INTO MEMBERSHIP
    (member_id, plan_id, start_date, end_date, membership_status)
VALUES
    (1, 1, DATE '2026-01-01', DATE '2026-01-31', 'Expired');

INSERT INTO MEMBERSHIP
    (member_id, plan_id, start_date, end_date, membership_status)
VALUES
    (1, 3, DATE '2026-09-01', DATE '2027-08-31', 'Active');

INSERT INTO MEMBERSHIP
    (member_id, plan_id, start_date, end_date, membership_status)
VALUES
    (2, 2, DATE '2026-08-01', DATE '2026-10-31', 'Pending');
```

```sql

INSERT INTO MEMBERSHIP
    (member_id, plan_id, start_date, end_date, membership_status)
VALUES
    (3, 1, DATE '2026-07-01', DATE '2026-07-31', 'Expired');


/* ============================================================
   5. FITNESS_CLASS
   Insert four fitness classes.
   Class 4 has no bookings.
   ============================================================ */

INSERT INTO FITNESS_CLASS
    (class_name, trainer_id, capacity, scheduled_at)
VALUES
    ('Strength Basics', 1, 20,
     TIMESTAMP '2026-10-15 09:00:00');

INSERT INTO FITNESS_CLASS
    (class_name, trainer_id, capacity, scheduled_at)
VALUES
    ('Morning Yoga', 2, 25,
     TIMESTAMP '2026-10-16 08:30:00');

INSERT INTO FITNESS_CLASS
    (class_name, trainer_id, capacity, scheduled_at)
VALUES
    ('Cardio Blast', 3, 20,
     TIMESTAMP '2026-10-16 17:00:00');

INSERT INTO FITNESS_CLASS
    (class_name, trainer_id, capacity, scheduled_at)
VALUES
    ('Pilates Introduction', NULL, 12,
     TIMESTAMP '2026-10-17 10:00:00');


/* ============================================================
   6. CLASS_BOOKING
   Insert four bookings.
   Member 1 has two bookings.
   Member 4 has zero bookings.
   Class 4 has zero bookings.
   ============================================================ */

INSERT INTO CLASS_BOOKING
    (member_id, class_id, booking_date, attendance_status)
VALUES
    (1, 1, DATE '2026-10-01', 'Attended');

INSERT INTO CLASS_BOOKING
    (member_id, class_id, booking_date, attendance_status)
```

```sql
VALUES
    (1, 2, DATE '2026-10-02', 'Booked');

INSERT INTO CLASS_BOOKING
    (member_id, class_id, booking_date, attendance_status)
VALUES
    (2, 1, DATE '2026-10-03', 'Absent');

INSERT INTO CLASS_BOOKING
    (member_id, class_id, booking_date, attendance_status)
VALUES
    (3, 3, DATE '2026-10-04', 'Cancelled');


/* ============================================================
   7. PAYMENT
   Insert four payments.
   Membership 4 has no payments.
   Include Successful, Pending, and Failed statuses.
   Member 1 has two successful payments.
   ============================================================ */

INSERT INTO PAYMENT
    (membership_id, payment_date, amount,
     payment_method, payment_status)
VALUES
    (1, DATE '2026-01-01', 40, 'Card', 'Successful');

INSERT INTO PAYMENT
    (membership_id, payment_date, amount,
     payment_method, payment_status)
VALUES
    (1, DATE '2026-01-15', 20, 'Cash', 'Successful');

INSERT INTO PAYMENT
    (membership_id, payment_date, amount,
     payment_method, payment_status)
VALUES
    (2, DATE '2026-09-01', 100, 'Bank Transfer', 'Pending');

INSERT INTO PAYMENT
    (membership_id, payment_date, amount,
     payment_method, payment_status)
VALUES
    (3, DATE '2026-08-01', 105, 'Card', 'Failed');


/* ============================================================
   SAVE ALL INSERTED RECORDS
   ============================================================ */

COMMIT;
```

```sql

/* ============================================================
   END OF SECTION 3: INSERT SAMPLE DATA
   ============================================================ */


/* ============================================================
   Section 4: QUERIES
   Retrieve and analyze gym membership management data.
   ============================================================ */

/* Query 1: Retrieve membership details including member name,
   plan name, start date, and membership status. */

SELECT
    m.membership_id,
    mb.full_name AS member_name,
    mp.plan_name,
    m.start_date,
    m.membership_status
FROM MEMBERSHIP m
INNER JOIN MEMBER mb
    ON m.member_id = mb.member_id
INNER JOIN MEMBERSHIP_PLAN mp
    ON m.plan_id = mp.plan_id;









/* Query 2: Display all members and their memberships,
   including members without any memberships. */

SELECT
    mb.member_id,
    mb.full_name AS member_name,
    NVL(TO_CHAR(m.membership_id), 'No Membership') AS membership_id,
    NVL(TO_CHAR(m.start_date, 'YYYY-MM-DD'), 'No Membership') AS start_date,
    NVL(m.membership_status, 'No Membership') AS membership_status
FROM MEMBER mb
LEFT JOIN MEMBERSHIP m
    ON mb.member_id = m.member_id;
```

```sql

/* Query 3: Display all fitness classes and their bookings,
   including classes without any bookings. */

SELECT
    fc.class_id,
    fc.class_name,
     NVL(TO_CHAR(cb.booking_id), 'No Bookings') AS booking_id,
    NVL(TO_CHAR(cb.member_id), 'No Member') AS member_id,
    NVL(TO_CHAR(cb.booking_date, 'YYYY-MM-DD'), 'No Booking Date') AS
booking_date,
    NVL(cb.attendance_status, 'No Status') AS attendance_status
FROM CLASS_BOOKING cb
RIGHT JOIN FITNESS_CLASS fc
    ON cb.class_id = fc.class_id;









/* Query 4: Display all trainers and their fitness classes,
   including trainers without any assigned classes. */

SELECT
    t.trainer_id,
    t.full_name AS trainer_name,
    NVL(fc.class_name, 'No Assigned Classes') AS class_name,
    NVL(TO_CHAR(fc.capacity), 'N/A') AS class_capacity,
    NVL(TO_CHAR(fc.scheduled_at, 'YYYY-MM-DD HH24:MI'), 'N/A') AS scheduled_at
FROM TRAINER t
LEFT JOIN FITNESS_CLASS fc
    ON t.trainer_id = fc.trainer_id;









/* Query 5: Display each membership plan with its membership
   count and average plan price, ordered by average price
   in descending order. */

SELECT
    mp.plan_name,
    COUNT(m.membership_id) AS total_memberships,
    AVG(mp.price) AS average_plan_price
```

```sql
FROM MEMBERSHIP_PLAN mp
INNER JOIN MEMBERSHIP m
    ON mp.plan_id = m.plan_id
GROUP BY
    mp.plan_id,
    mp.plan_name
ORDER BY
    average_plan_price DESC;









/* Query 6: Display each member's total class bookings,
   including members without bookings, ordered by booking
   count in descending order and member name ascending. */

SELECT
    mb.full_name AS member_name,
    COUNT(cb.booking_id) AS total_bookings
FROM MEMBER mb
LEFT JOIN CLASS_BOOKING cb
    ON mb.member_id = cb.member_id
GROUP BY
    mb.member_id,
    mb.full_name
ORDER BY
    total_bookings DESC,
    member_name ASC;









/* Query 7: Calculate the total revenue from successful
   payments for each membership plan. */

SELECT
    mp.plan_name,
    SUM(p.amount) AS total_revenue
FROM MEMBERSHIP_PLAN mp
INNER JOIN MEMBERSHIP m
    ON mp.plan_id = m.plan_id
INNER JOIN PAYMENT p
    ON m.membership_id = p.membership_id
WHERE p.payment_status = 'Successful'
```

```sql
GROUP BY
    mp.plan_id,
    mp.plan_name
ORDER BY
    total_revenue DESC;




/* Query 8: Display each fitness class and its total number
   of bookings, including classes without bookings, ordered
   by booking count in descending order. */

SELECT
    fc.class_name,
    COUNT(cb.booking_id) AS total_bookings
FROM FITNESS_CLASS fc
LEFT JOIN CLASS_BOOKING cb
    ON fc.class_id = cb.class_id
GROUP BY
    fc.class_id,
    fc.class_name
ORDER BY
    total_bookings DESC;









/* Query 9: Display each membership ID and classify its
   status as Active, Expired, or Pending. */

SELECT
    membership_id,
    CASE membership_status
        WHEN 'Active'  THEN 'Active Membership'
        WHEN 'Expired' THEN 'Expired Membership'
        WHEN 'Pending' THEN 'Pending Membership'
        ELSE 'Unknown Status'
    END AS status_description
FROM MEMBERSHIP
ORDER BY membership_id ASC;
```

```sql


/* Query 10: Display each member's contact information,
   prioritizing phone number, then email, and showing
   a message if neither is available. */

SELECT
    full_name AS member_name,
    COALESCE(phone, email, 'No contact details') AS contact_details
FROM MEMBER
ORDER BY member_id ASC;









/* Query 11: Display each membership and its total payment
   amount, including memberships without any payments. */

SELECT
    m.membership_id,
    NVL(SUM(p.amount), 0) AS total_amount_paid
FROM MEMBERSHIP m
LEFT JOIN PAYMENT p
    ON m.membership_id = p.membership_id
GROUP BY
    m.membership_id
ORDER BY
    m.membership_id ASC;









/* Query 12: Display each trainer and the total number of
   assigned fitness classes, including trainers without
   any classes, ordered by class count in descending order. */

SELECT
    t.full_name AS trainer_name,
    COUNT(fc.class_id) AS total_classes
FROM TRAINER t
LEFT JOIN FITNESS_CLASS fc
    ON t.trainer_id = fc.trainer_id
GROUP BY
```

```sql
    t.trainer_id,
    t.full_name
ORDER BY
    total_classes DESC,
    trainer_name ASC;









/* Query 13: Display payment details with status labels,
   ordered by payment date from newest to oldest. */

SELECT
    payment_id,
    membership_id,
    payment_method,
    amount,
    payment_date,
    CASE
        WHEN payment_status = 'Successful' THEN 'Completed'
        WHEN payment_status = 'Pending' THEN 'Pending'
        WHEN payment_status = 'Failed' THEN 'Not completed'
    END AS status_label
FROM PAYMENT
ORDER BY payment_date DESC;









/* Query 14: Calculate payment statistics for each payment
   method, including total, average, minimum, maximum,
   and number of payments. */

SELECT
    payment_method,
    SUM(amount) AS total_amount,
    AVG(amount) AS average_amount,
    MIN(amount) AS minimum_amount,
    MAX(amount) AS maximum_amount,
    COUNT(payment_id) AS total_payments
FROM PAYMENT
GROUP BY payment_method
ORDER BY total_amount DESC;
```

```sql
/* Query 15: Display members with at least two successful
   payments, including their payment count and total amount,
   ordered by total successful payments in descending order. */

SELECT
    mb.full_name AS member_name,
    COUNT(p.payment_id) AS successful_payment_count,
    SUM(p.amount) AS total_successful_payments
FROM MEMBER mb
INNER JOIN MEMBERSHIP m
    ON mb.member_id = m.member_id
INNER JOIN PAYMENT p
    ON m.membership_id = p.membership_id
WHERE p.payment_status = 'Successful'
GROUP BY
    mb.member_id,
    mb.full_name
HAVING COUNT(p.payment_id) >= 2
ORDER BY
    total_successful_payments DESC;
```